



PYTHON POUR LE DEVOPS

Ali Koudri - ali.koudri@janus-academy.fr

Python pour le DevOps

Fil rouge : Beacon - une CLI de supervision construite sur 2 jours

Ali Koudri

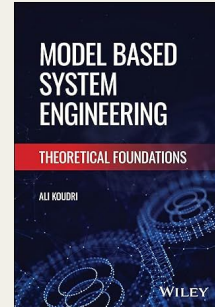
- PhD en informatique
- 25+ ans en ingénierie système et logicielle, modélisation, performance, sécurité
- IRT SystemX - Thales - CEA
- Janus Academy : formation & conseil indépendant
- Approche cross-stack : front, back, ci/cd, observabilité

Publications

Model Based Systems Engineering: Theoretical Foundations

Posture pédagogique

Démarche systémique, mesure avant optimisation, capitalisation transposable



L'application Beacon : Vue d'ensemble

Une CLI de supervision légère : elle lit une liste de cibles, les sonde, agrège leur état et l'expose. Voici ses composants, de l'entrée à la sortie.



Construite incrémentalement : M1 - cœur local · M2 - réseau, secrets & cloud · M3 - conteneur, observabilité & CI. Empaquetée en image, déployée par Ansible, intégrée en pipeline.



PYTHON POUR LE DEVOPS

Module 1 Partie 1

Du script au programme structuré

Fil rouge : Beacon - une CLI de supervision construite sur 2 jours

Pourquoi Python pour le DevOps

Le DevOps automatise des opérations sur des systèmes réels. Encore faut-il le bon langage pour cette automatisation. Trois constats expliquent la place qu'y a prise Python.



Tout est programmable

Services, cloud, conteneurs : chaque brique expose une API. L'ops devient du code.



bash plafonne vite

Dès qu'il faut de la logique, du parsing, des tests ou de la réutilisation, le shell montre ses limites.



Python, langage de colle

Lisible, batteries incluses, écosystème d'outils d'infra immense : il s'est imposé comme lingua franca.

La promesse et sa limite

La promesse

- Automatiser de vraies tâches DevOps
 - Avec un outil structuré, testé, livré
 - Construit de bout en bout : Beacon
- Du script local au service intégré en CI

La limite, assumée

- On n'apprend pas à « tout faire en Python »
 - Mais où Python est le bon outil...
 - ...et où bash, YAML, Terraform le sont davantage
- La compétence visée inclut le discernement

Au programme - Module 1 Partie 1

0

Mise en route

Calibrage, règles du jeu, présentation de Beacon

20 min

1.1

Du script au programme structuré

Fichiers I/O, pathlib, exceptions, modules

50 min

L1

Lab - Beacon lit sa config

Socle + extension, auto-rythmé

40 min

1.2

Interagir avec le système

subprocess, os/env, Python vs bash

50 min

L2

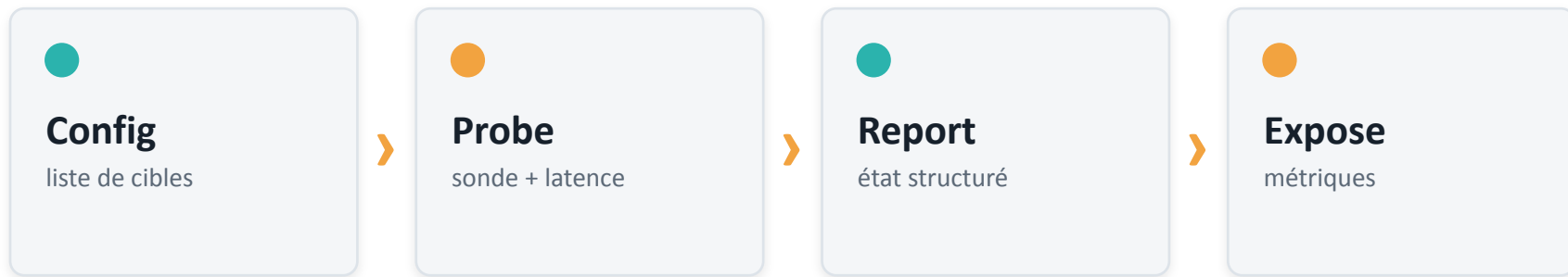
Lab - Beacon sonde en local

Socle + extension, auto-rythmé

40 min

Beacon - ce que l'on construit ensemble

Une CLI de supervision légère. Elle lit une liste de cibles, les sonde, structure leur état, l'expose en métriques et finit déployée et intégrée à une chaîne CI.



Module 1 Partie 1 : le cœur local de Beacon - lecture de config + sonde en local, en code structuré et propre.

Où l'on va sur deux jours

M1

Python opérationnel

Du script au paquet testé. Beacon sonde en local.

M2

Réseau, cloud, infra

API REST, secrets, SDK cloud, Ansible. Beacon sort de la machine.

M3

Production

Conteneurs, observabilité, CI (+ serverless optionnel).



MODULE 0

Mise en route

Règles du jeu

 Socle obligatoire

Chaque lab a un objectif minimal que tout le monde atteint. C'est le contrat.

 Extensions

Pour les plus rapides : on va plus loin, jamais évalué, jamais « dû ».

 Labs auto-rythmés

Vous avancez à votre vitesse. Pas de course de groupe.

 Checkpoints

À chaque fin de partie, on resynchronise sur une solution de référence.

À la fin de cette 1ère partie, vous saurez...

- 1 Lire et valider un fichier de configuration proprement (I/O, pathlib).
- 2 Gérer les erreurs sans les avaler - échouer tôt et clairement.
- 3 Découper un script en modules à responsabilité unique.
- 4 Lancer une commande système depuis Python avec subprocess.
- 5 Décider quand Python est le bon outil... et quand bash suffit.
- 6 Produire un résultat de sonde structuré et réutilisable.



BLOC 1.1

Du script au programme structuré

Python, l'automatisation et le DevOps

1991

Python (Guido van Rossum)

Un langage pensé pour la lisibilité « une façon évidente de faire ».

2000s

Du scripting système

Python supplante peu à peu Perl pour l'administration et la glue.

2009

Le mouvement DevOps

Les DevOpsDays (Patrick Debois, Gand) : briser le mur Dev / Ops.

2012

Ansible (Michael DeHaan)

Un config management écrit en Python, lisible et extensible.

2010s

Infrastructure as Code

L'infrastructure se décrit, se versionne et se teste comme du code.

Le DevOps, une question avant des outils

Avant d'être une pile d'outils, le DevOps répond à une fracture organisationnelle : développer vite et garder stable sont deux objectifs en tension, longtemps confiés à deux équipes séparées.

Loi de Conway (1967)

Un système reflète la structure de communication de l'organisation qui le produit. Un silo Dev / Ops produit des systèmes silotés, et fragiles à la frontière.

Ce que cela implique pour ce cours : l'automatisation Python est l'outil qui rend le flux Dev↔Ops fiable et répétable. Mais l'enjeu premier est organisationnel ; l'outil ne fait que servir une intention.

De l'impératif au déclaratif

Impératif

- On dit COMMENT, étape par étape
 - Les premiers scripts d'administration
- Puissant, mais fragile à rejouer

Déclaratif

- On dit QUOI : l'état désiré
 - L'outil converge (Ansible, Terraform, K8s)
- Rejouable en confiance

Idempotence : rejouer mène toujours au même état (une forme de point fixe). Python vit des deux côtés : il écrit la logique impérative et étend les outils déclaratifs. Comprendre ce glissement, c'est comprendre l'outillage moderne.

Coder l'infrastructure, c'est expliciter le tacite

« Infrastructure as code » transforme un savoir-faire tacite (ce que l'administrateur sait faire) en un artefact explicite, versionné, partageable. C'est un gain réel, mais qui a une limite.

Ce que le code capture

- La procédure, reproductible
 - Auditable et transmissible
- Le geste, détaché de la personne

Ce qu'il ne capture pas

- Le pourquoi, la compréhension
 - Un runbook exécute ; il ne raisonne pas
- L'outil amplifie le savoir, ne le remplace pas

Du jetable au durable

La tension

Combien de scripts d'automatisation meurent dès qu'un collègue les reprend, ou qu'ils doivent tourner ailleurs que sur le poste de leur auteur ?

Comment cette partie y répond

On transforme le script jetable en outil : structuré, à la configuration externalisée, robuste aux erreurs, la base de tout le reste.

« Ça marche sur ma machine »

Le script jetable de 80 lignes finit toujours par devoir tourner ailleurs, être relu, être testé, évoluer. À ce moment-là, le monolithe coûte cher.

Script jetable

- Tout dans un fichier
 - Chemins en dur
 - Aucune gestion d'erreur
- Intestable

Outil structuré

- Modules à rôle unique
 - Config externalisée
 - Erreurs explicites
- Testable & packageable

Lire et écrire proprement

Toujours un context manager

- Le fichier se ferme même si une exception survient.

Toujours préciser l'encodage

- `encoding="utf-8"`, jamais l'encodage par défaut du système.

Itérer sur le fichier pour le ligne à ligne, pas `read()` puis `split`.

```
from pathlib import Path
```

```
p = Path("config/targets.yaml")  
with p.open(encoding="utf-8") as f:  
    raw = f.read()
```

with garantit la fermeture. `Path.read_text()` fait souvent l'affaire en une ligne.

pathlib plutôt que os.path

Manipuler des chemins comme des objets : lisible, portable Windows/Linux, testable.

```
from pathlib import Path

# l'opérateur / compose les chemins
cfg = Path("config") / "targets.yaml"

cfg.exists()          # -> bool
cfg.read_text(encoding="utf-8")
cfg.suffix             # -> '.yaml'
```

Pourquoi

- Pas de concaténation de strings
- Séparateurs gérés pour vous
- API riche : glob, parent, name...

Se mocke facilement en test

Lire la configuration des cibles

Beacon lit sa liste de cibles depuis un fichier YAML. On parse avec `safe_load` - jamais `load()`.

```
# targets.yaml
targets:
  - name: api
    host: 127.0.0.1
    port: 8080
```

```
import yaml

data = yaml.safe_load(cfg.read_text())
targets = data["targets"]
```

`safe_load()` refuse les objets Python arbitraires : pas d'exécution de code caché depuis un fichier de config.

Gérer les erreurs sans les attraper

Attrapez ce que vous savez traiter

- `FileNotFoundError`, `yaml.YAMLError` - pas un except nu.

N'attrapez jamais une erreur en silence

- loggez, enrichissez, et relancez si vous ne pouvez pas traiter.

Échouez tôt : config invalide = on s'arrête, message clair.

```
try:
    data = load_config(cfg)
except FileNotFoundError:
    raise ConfigError(f"absente: {cfg}")
except yaml.YAMLError as e:
    raise ConfigError(...) from e
```

Du monolithe aux modules

Une responsabilité par module. Chacun se teste et se relit isolément.

config.py



charge & valide la config

probe.py



sonde une cible, renvoie un état

report.py



met en forme les résultats

cli.py

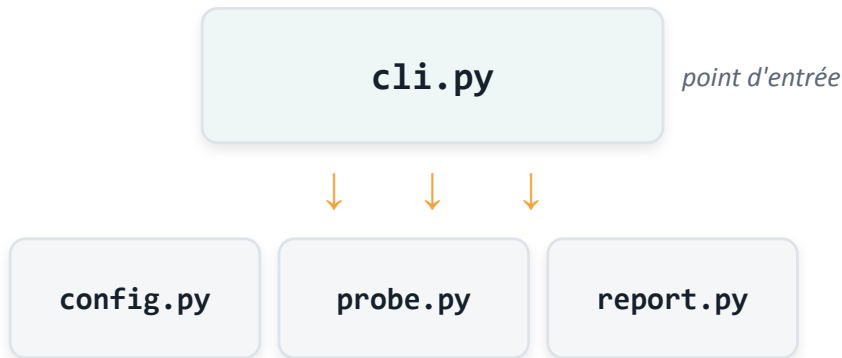


orchestre, parse les arguments

Règle simple : si vous ne savez pas où mettre une fonction, c'est que le découpage est flou.

Le piège des imports cycliques

En découpant en modules, un débutant fait souvent importer le point d'entrée par un module du bas. Résultat : un import cyclique, et une ImportError au chargement.



La règle

Le flux d'import descend. Les modules du bas n'importent jamais cli.py. Si l'un d'eux a besoin de remonter, c'est le signe d'un découpage à revoir.

Dernier recours seulement : un import local (dans la fonction) brise le cycle — mais traite le symptôme, pas la cause.

L'architecture cible du projet

beacon/

```
|— __init__.py
|— config.py      # load_config, validation
|— probe.py       # probe(target) -> Result
|— report.py      # format des résultats
|— cli.py         # point d'entrée
```

tests/

```
|— __init__.py
|— test_config.py # arrive La prochaine partie
```

On y arrive par étapes

- Lab 1 : config.py
- Lab 2 : probe.py
- Partie 2 : cli.py + tests

La cible n'est pas à écrire d'un coup - elle émerge.

Pydantic, le standard de la config moderne

Valider une configuration à coups de if devient vite ingérable. Pydantic le fait de façon déclarative, par les types, et produit des messages d'erreur précis.

```
from pydantic import BaseModel
from pydantic_settings import BaseSettings

class Target(BaseModel):
    name: str
    host: str
    port: int = 80

class Settings(BaseSettings):
    beacon_token: str    # lu depuis l'env
```

Pourquoi c'est devenu le standard

- Validation par les types, déclarative
 - Au cœur du Python moderne (FastAPI, etc.)
 - pydantic-settings : config via l'environnement
- Pont direct avec les secrets du J2

Dataclass vs Pydantic

La dataclass standard (Python natif) : Elle est idéale pour les structures de données internes à votre application (comme le ProbeResult qui voyage entre le module probe.py et le module report.py). Elle fait confiance au développeur et n'effectue aucune validation de type à l'exécution. Si vous passez un int là où un str est attendu, la dataclass l'acceptera sans broncher.

Le modèle Pydantic : Il est conçu pour sécuriser les frontières externes de votre application (lecture du fichier targets.yaml ou variables d'environnement). Pydantic va activement valider, convertir (coercion) et rejeter les données incorrectes à l'exécution en levant une erreur claire.

An orange circle containing the text "L1" in white, bold, sans-serif font.

L1

LAB 1

Beacon lit sa config

40 min · auto-rythmé · socle + extension

Beacon lit sa config

SOCLE - pour tous

- Lire un targets.yaml (cibles : name, host, port)
- Le parser avec safe_load
- Valider la structure minimale
- Gérer fichier absent / YAML invalide avec un message clair

Exposer load_config(path) -> list[Target]

EXTENSION - si vous avancez vite

- Supporter YAML + TOML
 - Valider par schéma avec pydantic
 - Champs optionnels avec valeurs par défaut
- Messages d'erreur pointant la ligne fautive



BLOC 1.2

Interagir avec le système

subprocess : lancer une commande

```
import subprocess

r = subprocess.run(
    ["ping", "-c", "1", host],
    capture_output=True,
    text=True,
    timeout=5,
)
r.returncode    # 0 = ok
```

Liste d'arguments, pas une chaîne

- shell=False par défaut = pas d'injection.

text=True

- stdout/stderr en str, pas en bytes.

timeout obligatoire

- une sonde qui pend bloque tout Beacon.

Les pièges de subprocess

 shell=True

Ouvre la porte à l'injection de commande. À bannir avec des entrées variables.

 check=True

Lève CalledProcessError si code $\neq$ 0. Pratique - mais à choisir consciemment.

 Oublier timeout

Le processus peut tourner indéfiniment. Toujours borner.

 Parser stdout fragile

Les sorties humaines changent. Préfère des formats stables (JSON, codes retour).

os & variables d'environnement

```
import os

# lire une variable, avec défaut
level = os.environ.get(
    "BEACON_LOG", "info")

# jamais de secret en dur :
token = os.environ["API_TOKEN"]
```

À retenir

- `.get(clé, défaut)` ne lève pas
- `[clé]` lève `KeyError` si absente - utile pour un secret requis
- L'environnement est le canal standard pour la config sensible

On y revient en détail au M2 (secrets).

Python ou bash ?

Python est un excellent langage de colle - pas la réponse universelle. Choisissez l'outil, pas le réflexe.

Plutôt Python

- Logique conditionnelle non triviale
- Parsing/transformation structurée
- Code à réutiliser et à tester

Portabilité multi-OS

Plutôt bash

- Enchaînement simple de commandes
- One-liner trivial et jetable
- Glue shell standard (pipe, redirections)

`curl | jq` peut battre 40 lignes de Python

Sonder une cible en local

Beacon ouvre une connexion TCP vers la cible et mesure la latence. Le résultat est traduit en statut.

```
import socket, time

start = time.perf_counter()
try:
    socket.create_connection(
        (host, port), timeout=2)
    status = "up"
except OSError:
    status = "down"
```

Du brut au statut

-  up - connexion établie
-  down - refus / timeout

La latence = perf_counter() après moins avant.

Un résultat de sonde structuré

Ne renvoie pas une string. Une dataclass rend le résultat explicite, typé et réutilisable (report, métriques...).

```
from dataclasses import dataclass

@dataclass
class ProbeResult:
    target: str
    status: str          # 'up' | 'down'
    latency_ms: float
    checked_at: str      # ISO 8601
```

Pourquoi typé

- Contrat clair entre modules
 - Sérialisable (JSON) pour le report
 - Facile à assérer en test
- Prêt pour les métriques du M3

An orange circle containing the text "L2" in white, bold, sans-serif font.

LAB 2

Beacon sonde en local

40 min · auto-rythmé · socle + extension

Beacon sonde en local

SOCLE - pour tous

- Sonder une cible locale (TCP) depuis la config du Lab 1
 - Mesurer la latence
 - Mapper le résultat en statut up / down
 - Renvoyer un ProbeResult structuré
- probe(target) -> ProbeResult

EXTENSION - si tu avances

- Timeout configurable par cible
 - Retries avec un court délai
 - Sonder toutes les cibles en séquence, proprement
- Agréger un mini-rapport (n up / n down)

Avant la partie 2

On resynchronise tout le monde sur une solution de référence. Chacun cale son code dessus avant de continuer.

Ce que vous devez avoir à ce stade :

- Un `load_config()` qui lit et valide `targets.yaml` et échoue clairement
- Un `probe()` qui renvoie un `ProbeResult` (up/down + latence)
- Deux modules séparés : `config.py` et `probe.py`

Pas encore de CLI ni de tests - c'est l'objet de la partie 1.

Ce module

Acquis en partie 1

- I/O et pathlib propres
- Erreurs explicites
- Découpage en modules
- subprocess & environnement

Résultat de sonde structuré

Prochaine partie

- Une vraie CLI (argparse)
- venv, pyproject.toml, uv
- Packaging : beacon en commande
- pytest + mocking

Tester sans toucher le réseau réel



Fin de la partie 1

Beacon sait lire sa config et sonder en local. La prochaine partie : on en fait un outil packagé et testé.



PYTHON POUR LE DEVOPS

Module 1 · Partie 2

CLI, packaging, tests

Beacon devient un outil installable et testé

Au programme - Module 1 Partie 2

1.3

CLI, environnement, packaging

argparse, venv, pyproject.toml, uv, entry point

55 min

L3

Lab - Beacon devient une vraie CLI

Socle + extension, auto-rythmé

45 min

1.4

Tester du code qui touche le système

pytest, fixtures, mocking

55 min

L4

Lab - Beacon testé

Socle + extension, auto-rythmé

45 min



Checkpoint fin de M1

Resynchro sur la solution de référence

15 min

Du brouillon au livrable

La tension

Un outil qu'on ne sait ni installer ailleurs, ni tester, est un outil qu'on ne déploiera jamais sereinement.

Ce que cette demi-journée y répond

CLI, environnement reproductible, packaging et tests : Beacon devient une commande installable et protégée par un filet.

Où en est Beacon

Acquis en partie 1

- `config.py` - lit et valide la config
 - `probe.py` - sonde une cible, renvoie un `ProbeResult`
 - Code en modules à responsabilité unique
- Erreurs explicites (`ConfigError`)

La prochaine partie

- Emballer le tout derrière une CLI
 - Rendre les dépendances reproductibles
 - Installer beacon comme une commande
- Tester sans dépendre du réseau réel

À la fin du module 1

- 1 Exposer Beacon en ligne de commande avec argparse.
- 2 Isoler les dépendances dans un environnement virtuel.
- 3 Déclarer le projet et ses dépendances dans pyproject.toml.
- 4 Installer beacon comme une commande via un entry point.
- 5 Écrire des tests pytest lisibles, avec fixtures.
- 6 Mock la frontière système pour tester sans réseau réel.



BLOC 1.3

CLI, environnement, packaging

argparse : exposer une CLI

Une CLI explicite remplace les variables modifiées à la main dans le script. L'usage devient documenté et vérifiable.

```
import argparse

def parse_args():
    p = argparse.ArgumentParser(
        prog="beacon")
    p.add_argument(
        "--config", type=Path,
        required=True)
    return p.parse_args()
```

argparse offre

- --help généré automatiquement
- Typage et conversion des arguments
- Erreurs d'usage claires (exit 2)

Aucune dépendance externe

Des sous-commandes pour grandir

Beacon aura plusieurs actions : check, report... Les sous-commandes structurent la CLI sans la rendre illisible.

```
p = argparse.ArgumentParser(prog="beacon")
sub = p.add_subparsers(dest="cmd")

# beacon check --config ...
chk = sub.add_parser("check")
chk.add_argument("--config", type=Path,
                 required=True)
```

Bon réflexe

Une sous-commande = une intention claire. Évite la CLI fourre-tout pilotée par dix drapeaux.

cli.py orchestre le tout

```
def main():
    args = parse_args()
    try:
        targets = load_config(args.config)
    except ConfigError as e:
        print(e, file=sys.stderr)
        raise SystemExit(2)
    results = [probe(t) for t in targets]
    render(results)
    raise SystemExit(0 if all_up else 1)
```

La CLI est mince

- Elle parse, appelle le cœur, gère la sortie
- Aucune logique métier ici
- ConfigError présentée proprement, pas de stack trace

Le code de sortie porte le verdict

Les codes de sortie comptent

En DevOps, un outil est jugé par son code de retour : c'est lui que la CI et les scripts lisent. Soignez-le.

0

Tout va bien - toutes les cibles up

1

Échec fonctionnel - au moins une cible down

2

Erreur d'usage ou de configuration

Une convention stable rend Beacon scriptable et intégrable en CI dès le M3.

venv : isoler les dépendances

Chaque projet ses dépendances, sans polluer le système ni les autres projets. Le minimum vital de reproductibilité.

```
# créer puis activer un environnement
python -m venv .venv
source .venv/bin/activate

# les installations sont locales au projet
pip install pyyaml
```

Pourquoi

- Pas de conflit de versions entre projets
- Reproductible sur une autre machine / en CI

Se jette et se recrée sans risque

Environnements & dépendances : standardiser uv

Plutôt qu'introduire le module venv natif puis uv séparément, on standardise uv dès le départ : un seul outil pour l'environnement, les dépendances et l'exécution.

```
# créer l'environnement (remplace python -m venv)
uv venv

# installer le projet et ses dépendances
uv pip install -e .

# exécuter sans activation manuelle
uv run beacon check --config config/targets.yaml
```

Pourquoi standardiser

- uv venv remplace python -m venv, en bien plus rapide (Rust)
- Un seul outil : venv + pip + pip-tools réunis
- Résolution déterministe (lockfile)

Le venv natif reste bon à connaître : présent partout, zéro install

Déclarer le projet : pyproject.toml

Le fichier standard qui décrit le projet, ses dépendances et sa commande. La source de vérité du packaging Python moderne.

```
[project]
name = "beacon"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = ["pyyaml"]

[project.scripts]
beacon = "beacon.cli:main"
```

Un seul fichier

- Métadonnées
 - Dépendances
 - Commande exposée
- Outils (ruff, mypy...) au M3

uv : l'outillage moderne

```
# uv remplace venv + pip + pip-tools
uv venv
uv pip install -e .
uv run beacon check --config config/targets.yaml
```

Atouts

- Très rapide (écrit en Rust)
 - Résolution déterministe
- Un seul outil à apprendre

Point de vigilance - uv, Poetry, PDM, ou pip + venv classique font tous le travail. L'important n'est pas l'outil mais d'en choisir un, de le verrouiller (lockfile) et de s'y tenir dans l'équipe. Beacon utilisera uv ; vos projets feront leur choix.

De module à commande

Grâce à [project.scripts], l'installation crée un exécutable beacon qui appelle cli:main. Plus besoin de python -m.

```
# après installation éditable
uv pip install -e .

# beacon est maintenant une commande
beacon check --config config/targets.yaml
echo $?      # le code de sortie de Beacon
```

Le résultat

Un outil installable comme n'importe quel binaire - prêt à être conteneurisé au M3.

L3

LAB 3

Beacon devient une vraie CLI

45 min · auto-rythmé · socle + extension

Beacon devient une vraie CLI

SOCLE - pour tous

- Exposer beacon check --config via argparse
 - Orchestrer config + probe dans cli.py
 - Créer un venv et installer en éditable
 - Déclarer l'entry point dans pyproject.toml
- Lancer beacon comme une commande

EXTENSION - pour aller plus loin

- Codes de sortie 0 / 1 / 2 cohérents
 - Sous-commande report en plus de check
 - Option --format text|json
- Construire une wheel et l'installer



BLOC 1.4

Tester du code qui touche le système

Un script non testé est un risque

En DevOps, le code automatise des actions sur des systèmes réels. Le test est le filet qui permet de modifier sans casser - et la condition d'une CI utile.



Confiance pour changer

Refactorer sans peur de régression silencieuse.



Documentation vivante

Un test montre l'usage attendu, mieux qu'un commentaire.



Pré-requis de la CI

Sans tests, le pipeline du M3 ne protège rien.

pytest : les bases

Des fonctions `test_` et un simple `assert`. Pas de classe, pas de cérémonie.

```
# tests/test_config.py
def test_load_config_ok(tmp_path):
    cfg = tmp_path / "t.yaml"
    cfg.write_text(
        "targets:\n  - name: a\n"
        "    host: h\n    port: 1\n")
    targets = load_config(cfg)
    assert len(targets) == 1
```

`tmp_path`

Fixture intégrée : un répertoire temporaire propre par test. Idéal pour les fichiers de config, sans toucher au disque réel.

Fixtures : factoriser la mise en place

Une fixture prépare un contexte réutilisable. Les tests la reçoivent en argument, sans duplication.

```
import pytest

@pytest.fixture
def sample_cfg(tmp_path):
    p = tmp_path / "t.yaml"
    p.write_text(VALID_YAML)
    return p

def test_ok(sample_cfg):
    assert load_config(sample_cfg)
```

Garder simple

- Une fixture = un contexte clair
 - Réutilisée par plusieurs tests
- Évite le copier-coller fragile

Tester probe() sans réseau réel

Un test qui ouvre une vraie connexion réseau n'est pas un test unitaire : il est lent, dépend de l'extérieur, et échoue de façon aléatoire.

Réseau réel en test

- Lent
- Dépend d'un service externe
- Flaky : échoue sans raison

Inutilisable en CI

Frontière mockée

- Rapide et déterministe
- Aucun service requis
- On teste la logique, pas le réseau

Parfait pour la CI

Mocker la frontière système

On remplace l'appel réseau par un faux, le temps du test. monkeypatch (pytest) cible précisément la fonction à substituer.

```
def test_probe_up(monkeypatch):
    monkeypatch.setattr(
        "beacon.probe.socket.create_connection",
        lambda *a, **k: DummySock())

    r = probe(target)
    assert r.status == "up"
```

Le principe

On substitue `create_connection` là où `probe` l'utilise. La logique up/down est testée, le réseau ne l'est pas.

Tester aussi ce qui doit échouer

```
def test_missing_config():  
    with pytest.raises(ConfigError):  
        load_config(Path("nope.yaml"))  
  
def test_probe_down(monkeypatch):  
    monkeypatch.setattr(... raises OSError)  
    assert probe(t).status == "down"
```

Quoi mocker, quoi non

- Mocker : la frontière I/O (socket, subprocess)
 - Tester : la logique (parsing, mapping, codes)
- Ne pas mocker ce qu'on veut justement vérifier

Le chemin d'échec mérite autant de tests que le chemin nominal : c'est souvent là que les incidents naissent.

An orange circle containing the text "L4" in a bold, black, sans-serif font.

LAB 4

Beacon testé

45 min · auto-rythmé · socle + extension

Beacon testé

SOCLE - pour tous

- Tester load_config sur cas nominal (tmp_path)
 - Tester l'échec : fichier absent -> ConfigError
 - Tester probe up et down en mockant la socket
- Lancer la suite avec pytest, tout au vert

EXTENSION - pour aller plus loin

- Mesurer la couverture (pytest-cov)
 - Fixtures paramétrées (plusieurs configs)
 - Tester le YAML invalide -> ConfigError
- Tester les codes de sortie de la CLI

Fin du Module 1 Partie 2

Resynchronisation finale sur la solution de référence avant le module 2.

Beacon, à la fin de M1 :

- Lit et valide sa config, sonde des cibles locales (config.py, probe.py)
- S'utilise en ligne de commande : `beacon check --config ...`
- S'installe comme une commande (venv + pyproject.toml + entry point)

Est couvert par des tests qui ne dépendent pas du réseau réel

M2 - Beacon sort de la machine

Acquis du M1

- Code structuré et modulaire
- Système : I/O, subprocess
- CLI + packaging + commande

Tests avec mocking

Au M2

- Sonder des endpoints HTTP réels
- Gérer les secrets proprement
- Interroger le cloud (SDK)

Déployer Beacon via Ansible



Fin du Module 1 Partie 2

Beacon est un outil installable et testé. Au prochain module, il quitte la machine locale pour le réseau, le cloud et l'infrastructure.



PYTHON POUR LE DEVOPS

Module 2 - Partie 1

API REST & secrets

Beacon quitte la machine : il sonde le réseau et manipule des credentials

Au programme - M2 Partie 1

2.1

Consommer des API REST

httpx, requête, réponse, erreurs réseau

50 min

L5

Lab - Beacon sonde le web

Socle + extension, auto-rythmé

45 min

2.2

Secrets & credentials

Variables d'environnement, .env, gestionnaires

45 min

L6

Lab - Beacon gère ses secrets

Socle + extension, auto-rythmé

45 min



Checkpoint partie 2

Resynchro sur la solution de référence

15 min

Beacon, au sortir de M1

Acquis du M1

- Lit et valide sa config
 - Sonde des cibles locales (TCP)
 - S'utilise en CLI, s'installe en commande
- Couvert par des tests sans réseau réel

Objectif de la partie 1

- Sonder de vrais endpoints HTTP
 - Mesurer latence applicative et code statut
 - Manipuler des secrets proprement
- Sonder une cible authentifiée

Sortir de la machine

La tension

Le port répond, donc le service va bien ? Pas si vite ; et le token qui ouvre la porte, où le ranger sans le faire fuiter ?

Comment cette partie y répond

Sonder le réseau réel (HTTP), gérer les erreurs qui en découlent, et manipuler des secrets sans jamais les exposer.

À la fin de la partie 1

- 1 Émettre une requête HTTP avec httpx et lire sa réponse.
- 2 Gérer proprement les erreurs réseau (timeout, refus, statut).
- 3 Faire sonder des endpoints HTTP réels à Beacon.
- 4 Choisir entre requests et httpx selon le besoin.
- 5 Garder tout secret hors du code et du dépôt.
- 6 Injecter un credential via l'environnement pour une cible authentifiée.



BLOC 2.1

Consommer des API REST

Du port ouvert à la santé applicative

Une sonde TCP dit seulement « le port répond ». Une sonde HTTP dit « le service répond correctement » - c'est ce qui intéresse l'exploitation.

Sonde TCP (M1)

- Le port accepte une connexion
- Ne dit rien de l'application

Un service planté peut sembler up

Sonde HTTP (M2)

- Code de statut (2xx / 4xx / 5xx)
 - Latence applicative réelle
- Peut vérifier un endpoint /health

httpx : émettre une requête

```
import httpx

# timeout TOUJOURS explicite
r = httpx.get(url, timeout=5.0)

r.status_code      # 200
r.json()           # corps décodé
r.elapsed          # durée (timedelta)
```

Réflexes

- Sans timeout, une sonde peut pendre indéfiniment
- Un Client réutilisable mutualise les connexions

API quasi identique à requests

httpx : un Client réutilisable (pooling)

`httpx.get()` ouvre une connexion, fait la requête, la ferme, à chaque appel. Pour une sonde isolée, c'est acceptable ; pour un outil qui enchaîne les requêtes, c'est du gaspillage.

```
# une connexion rouverte à chaque appel
for t in targets:
    r = httpx.get(t.url, timeout=5)
```

```
# connexions réutilisées (pooling)
with httpx.Client(timeout=5) as c:
    for t in targets:
        r = c.get(t.url)
```

Pour un outil de monitoring : un Client partagé sur tout un cycle de sondes évite de répéter le handshake TCP/TLS, et garantit la fermeture propre des connexions en sortie de bloc.

Gérer les erreurs réseau

Le réseau échoue : timeout, refus de connexion, DNS, statut 5xx. Chaque cas doit se traduire en un statut, jamais en crash.

```
try:
    r = httpx.get(url, timeout=5.0)
    r.raise_for_status()
    status = "up"
except httpx.HTTPStatusError:
    status = "down"    # 4xx / 5xx
except httpx.RequestError:
    status = "down"    # timeout, DNS, refus
```

probe_http : sonder un service

```
def probe_http(target):  
    start = time.perf_counter()  
    try:  
        r = httpx.get(target.url,  
                      timeout=target.timeout)  
        ok = r.status_code < 400  
    except httpx.RequestError:  
        ok = False  
    ms = (time.perf_counter()-start)*1000  
    return ProbeResult(target.name,  
                       "up" if ok else "down", ms, now())
```

Même contrat

- Renvoie le même ProbeResult que la sonde TCP
- Le report et les métriques n'ont rien à changer

Le Target gagne un champ url

requests ou httpx ?

Les deux consomment des API REST avec une API quasi identique. Le choix se fait sur le besoin réel, pas sur la mode.

requests

- Standard de fait, ubiquitaire
- Synchrone, simple

Parfait pour quelques requêtes

httpx

- Synchrone ET asynchrone
 - HTTP/2, Client réutilisable
- Idéal pour sonder N cibles en parallèle (extension du lab)

L5

LAB 5

Beacon sonde le web

45 min · auto-rythmé · socle + extension

Beacon sonde le web

SOCLE - pour tous

- Ajouter un champ url au Target
 - Écrire probe_http() avec httpx
 - Traduire timeout / refus / statut en up ou down
 - Mesurer la latence applicative
- Agréger un rapport (n up / n down)

EXTENSION - pour aller plus loin

- Sondage concurrent (httpx async ou futures)
 - Backoff sur échec avant de conclure down
 - Vérifier un corps /health attendu
- Distinguer down (réseau) et degraded (lent)



BLOC 2.2

Secrets & credentials

Un secret en dur est un secret fuité

Sonder une cible authentifiée exige un token. Le mettre dans le code, c'est le publier : Git conserve tout l'historique, même après suppression.

En dur dans le code

- Présent dans l'historique Git
- Copié dans chaque clone et image
- Impossible à faire tourner

Visible de toute l'équipe

Injecté à l'exécution

- Le code ne contient aucun secret
- Fourni par l'environnement
- Rotation et révocation possibles

Différent par environnement

L'environnement, canal des secrets

```
import os

# requis : échoue tôt si absent
token = os.environ["BEACON_TOKEN"]

# optionnel : avec défaut
level = os.environ.get("BEACON_LOG",
                       "info")
```

Le bon réflexe

- [clé] pour un secret requis : `KeyError` clair si oublié
 - Standard reconnu par tous les orchestrateurs
- Aucune dépendance

Fichiers .env : pratique, jamais commité

En local, un fichier .env évite d'exporter les variables à la main. Condition absolue : il n'entre jamais dans le dépôt.

```
# .env - listé dans .gitignore
BEACON_TOKEN=s3cr3t-de-dev

# au démarrage, en dev seulement
from dotenv import load_dotenv
load_dotenv() # peuple os.environ
```

Garde-fou

- .env dans .gitignore, toujours
- Un .env.example sans valeurs documente les clés attendues

En prod : pas de .env - l'orchestrateur fournit l'environnement

Les gestionnaires de secrets

En production, ni code ni .env : un gestionnaire dédié fournit le secret au runtime, avec audit, contrôle d'accès et rotation.



Injection au runtime

Le secret n'existe ni dans le code ni dans l'image - il est monté ou récupéré à l'exécution.



Rotation & révocation

Changer un secret compromis sans redéployer le code.



Audit & accès

Qui lit quel secret, quand - traçable et restreint.

Vault, AWS Secrets Manager, GCP Secret Manager... le principe prime sur l'outil.

Sonder une cible authentifiée

```
# le secret vient de l'environnement
token = os.environ["BEACON_TOKEN"]

headers = {
    "Authorization": f"Bearer {token}"
}
r = httpx.get(target.url, headers=headers,
              timeout=5.0)
```

Ne jamais logger

- Pas de `print(token)`, pas de token dans une URL loggée
 - Masquer dans les messages d'erreur
- Un secret loggé = un secret fuité

Ce qui ne va jamais dans le dépôt

 Tokens, clés d'API, mots de passe

Hors du code et de l'historique Git - sans exception.

 Fichiers .env réels

Dans .gitignore ; seul un .env.example sans valeurs est versionné.

 Secrets dans les logs / URLs

Masqués partout - messages d'erreur et traces compris.

 Identifiants de connexion

Fournis par l'environnement ou un gestionnaire, jamais figés.

L6

LAB 6

Beacon gère ses secrets

45 min · auto-rythmé · socle + extension

Beacon gère ses secrets

SOCLE - pour tous

- Sortir le token du code : le lire dans l'environnement
 - Échouer clairement s'il est absent
 - Sonder une cible authentifiée (header Bearer)
- Vérifier qu'aucun secret n'apparaît dans les logs

EXTENSION - pour aller plus loin

- Hiérarchie de config : défauts < fichier < env < CLI
 - Fonction de redaction des secrets dans les logs
 - Support .env via python-dotenv en dev
- Un .env.example documentant les clés

Avant la partie 2

Resynchronisation sur la solution de référence avant le bloc cloud.

Beacon, à ce stade :

- Sonde des endpoints HTTP réels et en tire un statut + une latence
- Gère timeout, refus et statut sans jamais planter
- Lit ses credentials dans l'environnement, jamais dans le code

Ne laisse aucun secret fuir dans les logs

M2 - Cloud & infrastructure

Acquis sur cette partie

- Sonde HTTP réelle (httpx)
 - Gestion des erreurs réseau
 - Secrets hors du code
- Cible authentifiée

La prochaine partie

- Interroger le cloud via un SDK
 - Remonter l'état de ressources
 - Le pont avec Ansible
- Déployer et planifier Beacon



Fin de la partie 1

Beacon sonde le réseau et manipule des secrets proprement. La prochaine partie, il interroge le cloud et se déploie via Ansible.



PYTHON POUR LE DEVOPS

Module 2 · Partie 2

Cloud & infrastructure

Beacon interroge le cloud, puis se déploie - avec la juste place pour Python

Au programme - Module 2 Partie 2

2.3

Interagir avec le cloud

SDK, ressources, SDK vs CLI vs Terraform

50 min

L7

Lab - Beacon regarde le cloud

Socle + extension, auto-rythmé

45 min

2.4

Le pont avec le config management

Ansible, modules Python, scheduling

50 min

L8

Lab - Déployer Beacon

Socle + extension, auto-rythmé

45 min



Checkpoint fin de M2

Resynchro sur la solution de référence

15 min

Beacon, après la partie 1

Acquis sur cette partie

- Sonde des endpoints HTTP réels
 - Gère les erreurs réseau sans planter
 - Lit ses secrets dans l'environnement
- Sonde des cibles authentifiées

Objectif de la partie 2

- Interroger l'état de ressources cloud
 - Tester ce code sans compte cloud réel
 - Déployer Beacon via Ansible
- Situer Python parmi les outils d'infra

Cloud, sans dogme

La tension

Le cloud expose tout par API. Faut-il pour autant tout piloter en Python ? Et où s'arrête le rôle du langage ?

Comment cette partie y répond

Lire l'état d'une infra via un SDK, déployer Beacon avec Ansible, et situer honnêtement Python parmi les outils d'infrastructure.

À la fin de la partie 2

- 1 Authentifier un SDK cloud sans jamais coder de clé en dur.
- 2 Lister et inspecter l'état de ressources par programme.
- 3 Faire remonter cet état dans le rapport de Beacon.
- 4 Tester du code cloud sans compte réel (mock).
- 5 Déployer et planifier Beacon avec un playbook Ansible.
- 6 Situer la place réelle de Python : SDK et extensions, pas l'orchestration.



BLOC 2.3

Interagir avec le cloud

Le SDK : l'API cloud en Python

Le SDK expose les services cloud comme des appels Python. On l'utilise pour LIRE l'état d'une infra et, au besoin, AGIR dessus - avec de la logique, des conditions, des tests.

Lire l'état

Décrire instances, buckets, files, et leur statut.

Agir par programme

Démarrer, taguer, nettoyer - avec une logique conditionnelle.

Intégrer & tester

Le tout dans un outil versionné, testé, réutilisable.

Exemple pris sur AWS (boto3) - la logique se transpose à Azure et GCP.

Authentifier le SDK proprement

```
import boto3

# credentials résolus dans l'ordre :
# env -> profil -> rôle IAM
ec2 = boto3.client(
    "ec2", region_name="eu-west-3")
```

Dans la continuité

- Mêmes règles que les secrets de la partie 1: aucune clé en dur
 - En prod : rôle IAM, pas de clé du tout
- Le SDK résout les credentials pour vous

Lister et inspecter des ressources

```
resp = ec2.describe_instances()

for res in resp["Reservations"]:
    for inst in res["Instances"]:
        inst["InstanceId"]
        inst["State"]["Name"]    # running...
```

Volumes réels

- Au-delà de quelques ressources : utiliser un paginator
- Filtrer côté API (Filters) plutôt qu'en Python

Gérer les erreurs de permission (ClientError)

Remonter l'état dans le rapport

```
def probe_cloud(ec2, instance_id):  
    r = ec2.describe_instance_status(  
        InstanceIds=[instance_id])  
    st = r["InstanceStatuses"]  
    state = (st[0]["InstanceState"]["Name"]  
             if st else "unknown")  
    return ResourceState(instance_id,  
                          state, now())
```

Une vue unifiée

- Beacon agrège l'état cloud à côté de ses sondes réseau
- Même esprit que ProbeResult : un état structuré

Beacon LIT l'infra ; il ne la provisionne pas

SDK, CLI cloud, ou Terraform ?

Trois outils, trois rôles. Les confondre mène à réécrire en Python ce qu'un outil dédié fait mieux.



SDK (boto3)

Lire et agir par programme, avec logique et tests. Le choix de Beacon.



CLI cloud (aws)

Commandes ponctuelles, exploration, scripts shell rapides.



Terraform / IaC

Provisionner et maintenir un état désiré déclaratif. Pas du Python.

Beacon lit l'état (SDK). Il ne crée pas d'infrastructure - c'est le rôle de Terraform.

Tester sans compte cloud réel

Comme la socket puis HTTP, le cloud est une frontière : on la mocke. moto simule les services AWS - tests rapides, déterministes, sans facture.

```
from moto import mock_aws

@mock_aws
def test_probe_cloud():
    ec2 = boto3.client("ec2", ...)
    # créer une instance fictive, sonder,
    # assert l'état remonté
```

Cohérence

- Même principe qu'au M1 : mocker la frontière, tester la logique
 - Aucun credential, aucune ressource réelle requise
- Utilisable en CI

L7

LAB 7

Beacon regarde le cloud

45 min · auto-rythmé · socle + extension

Beacon regarde le cloud

SOCLE - pour tous

- Authentifier boto3 (sans clé en dur)
 - Lister des ressources et lire leur état
 - Écrire probe_cloud -> ResourceState
 - L'intégrer au rapport de Beacon
- Tester avec moto, sans compte réel

EXTENSION - pour aller plus loin

- Pagination sur de gros volumes
 - Filtrage côté API par tag
 - Gestion fine des erreurs de permission
- Sur compte réel : un service sandbox au choix



BLOC 2.4

Le pont avec le config management

Pourquoi Ansible, en cours Python

Ansible est écrit en Python et s'étend en Python. C'est le pont le plus direct entre Python et le config management - bien plus que de piloter des outils via subprocess.



Le quoi est déclaratif

Un playbook YAML décrit l'état voulu, pas les étapes.



Le comment custom est Python

Modules et plugins sur mesure s'écrivent en Python.



Idempotent par conception

Rejouer le playbook converge vers le même état.

Un playbook qui déploie Beacon

```
- hosts: monitors
  tasks:
    - name: Installer Beacon
      ansible.builtin.pip:
        name: beacon
    - name: Planifier la sonde
      ansible.builtin.cron:
        name: beacon-check
        minute: "*/5"
        job: "beacon check --config ..."
```

Déclaratif

- On décrit l'état cible, pas la procédure
- Rejouable sans effet de bord (idempotent)

Le YAML n'est pas du Python : c'est voulu

Modules custom : du Python

Quand une tâche n'existe pas, on écrit un module Ansible : un programme Python qui reçoit des arguments et renvoie un résultat structuré. C'est là que les deux mondes se rejoignent.

```
from ansible.module_utils.basic import AnsibleModule

def main():
    m = AnsibleModule(argument_spec={...})
    # logique métier en Python
    m.exit_json(changed=True, result=...)
```

Planifier Beacon : cron, ou Python ?

cron / systemd timers

- Exécuter Beacon périodiquement sur un hôte
- L'OS gère le déclenchement, les logs, les redémarrages

Le bon choix par défaut

APScheduler (Python)

- Utile si l'ordonnancement vit DANS un process Python long
 - Pas le cas pour une sonde lancée périodiquement
- Ne pas réimplémenter cron en Python

Pour Beacon, cron via le playbook suffit. Honnêteté : choisir le mécanisme le plus simple qui marche.

Python parmi les outils d'infra

Python excelle comme langage de logique et de colle. Il ne remplace ni le déclaratif, ni l'orchestrateur, ni le provisioning.

● Python

Logique custom, SDK (lire/agir), modules Ansible, Beacon

● YAML

Déclaratif : playbooks Ansible, manifests Kubernetes

● Terraform / HCL

Provisionner l'infra, maintenir un état désiré

● bash / cron

Glue système, enchaînements simples, planification

An orange circle containing the text "L8" in white, bold, sans-serif font.

LAB 8

Déployer Beacon

45 min · auto-rythmé · socle + extension

Déployer Beacon

SOCLE - pour tous

- Écrire un playbook qui installe Beacon sur un hôte
- Déposer la config et planifier la sonde (cron)
- Vérifier l'idempotence : rejouer ne change rien

Hôte cible : une VM ou un conteneur

EXTENSION - pour aller plus loin

- Écrire un petit module ou filtre Ansible en Python
 - Injecter les secrets via l'environnement de l'hôte
 - Paramétrer par variables d'inventaire
- Gérer plusieurs hôtes (groupe monitors)

Fin du module 2

Resynchronisation finale sur la solution de référence avant le M3.

Beacon, à la fin du M2 :

- Sonde le réseau (TCP, HTTP) et l'état de ressources cloud
- Manipule secrets et credentials sans rien exposer
- Se teste sans réseau ni cloud réels (mocks)

Se déploie et se planifie via Ansible, de façon idempotente

M3 - Beacon en production

Acquis du M2

- Sonde HTTP réelle, secrets maîtrisés
 - Lecture de l'état cloud (SDK)
 - Déploiement via Ansible
- Python situé parmi les outils d'infra

Au M3

- Conteneuriser Beacon (Docker)
 - Exposer des métriques, logs structurés
 - Intégrer dans un pipeline CI
- Serverless (optionnel)



Fin du module 2

Beacon lit le cloud et se déploie via Ansible, sans surévaluer Python. Demain : conteneur, observabilité et CI - la mise en production.



PYTHON POUR LE DEVOPS

Module 3 · Partie 1

Conteneurs & observabilité

Beacon devient un service livrable et observable

Au programme - M3 partie 1

3.1

Conteneuriser

Dockerfile, build, run, bonnes pratiques

45 min

L9

Lab - Beacon en conteneur

Socle + extension, auto-rythmé

45 min

3.2

Observabilité

Logs structurés, métriques exposées

50 min

L10

Lab - Beacon observable

Socle + extension, auto-rythmé

45 min



Checkpoint partie 2

Resynchro sur la solution de référence

15 min

Beacon, au sortir du M2

Acquis des M1 et M2

- Sonde réseau (TCP, HTTP) et état cloud
 - Secrets et credentials maîtrisés
 - CLI packagée, testée (mocks)
- Déployable via Ansible

Objectif de la partie 1

- Empaqueter Beacon en image conteneur
 - Le livrer de façon reproductible
 - Le rendre observable : logs et métriques
- Exposer un endpoint /metrics

Bon pour la production

La tension

« Ça marche en test » ne suffit pas en production : il faut livrer à l'identique, et savoir ce que fait le service une fois lancé.

Comment cette partie y répond

Empaqueter Beacon en image reproductible, puis le rendre observable : logs structurés et métriques exposées.

À la fin de la partie 1

- 1 Écrire un Dockerfile correct pour une application Python.
- 2 Construire et exécuter Beacon en conteneur.
- 3 Appliquer les bonnes pratiques d'image (slim, non-root, layers).
- 4 Produire des logs structurés plutôt que des print.
- 5 Exposer des métriques Beacon via un endpoint `/metrics`.
- 6 Situer le rôle de chacun : Beacon expose, Prometheus collecte, Grafana affiche.



BLOC 3.1

Conteneuriser

L'image, artefact de livraison

Un conteneur emballe Beacon avec sa version de Python et ses dépendances. Ce qui tourne en test tourne à l'identique en production - l'image est l'unité de livraison.



Reproductible

Mêmes dépendances partout, du poste à la prod.



Portable

Tourne sur tout hôte doté d'un runtime conteneur.



Immuable

On déploie une image versionnée, pas un état mouvant.

Un Dockerfile pour Beacon

```
FROM python:3.12-slim
WORKDIR /app

# deps avant code : cache des layers
COPY pyproject.toml ./
COPY beacon/ ./beacon/
RUN pip install --no-cache-dir .

ENTRYPOINT ["beacon"]
CMD ["check", "--config", "/etc/beacon/targets.yaml"]
```

Construire et exécuter

```
# construire l'image versionnée
docker build -t beacon:0.1 .

# exécuter : config en volume, secret en env
docker run --rm \
  -e BEACON_TOKEN \
  -v ./config:/etc/beacon \
  beacon:0.1
```

Dans la continuité

- Config montée en volume, pas figée dans l'image
- Secret injecté via -e : mêmes règles qu'au M2

L'image reste générique et réutilisable

⚠ PowerShell ou CMD peuvent parfois nécessiter la syntaxe \${PWD}/config pour éviter des comportements étranges de montage de dossier vide.

PID 1 & SIGTERM dans le conteneur

Avec ENTRYPOINT ["beacon"], le process Python devient PID 1. Or, par défaut, Python n'installe pas de gestionnaire pour SIGTERM : le signal qu'envoie docker stop.

Le risque

Si Beacon tourne en boucle (métriques, sondage continu), il ignore SIGTERM. Docker attend 10 s puis tue brutalement — sans fermeture propre.

Alternative

docker run --init (ou tini) fournit un init léger qui gère signaux et processus zombies.

```
import signal, sys

def shutdown(signum, frame):
    client.close()    # fermeture propre
    sys.exit(0)

signal.signal(signal.SIGTERM, shutdown)
```

Un arrêt gracieux ferme le Client httpx, vide les logs et les métriques en attente, et libère les ressources, au lieu d'un kill -9.

Bonnes pratiques d'image Python



Base slim, version pinnée

python:3.12-slim plutôt que latest : léger et prévisible.



Utilisateur non-root

Créer et basculer sur un user dédié - surface d'attaque réduite.



Multi-stage build

Séparer la construction du runtime : image finale minimale.



.dockerignore

Ne pas copier .git, .venv, .env, tests dans l'image.

La juste place de Python ici

Conteneuriser, ce n'est pas « faire du Docker en Python ». Python vit DANS le conteneur ; Docker construit et exécute ; un orchestrateur coordonne.

Ce que fait Python

- Le code de Beacon, dans l'image
 - Packagé proprement (pyproject)
- Observable (bloc suivant)

Ce que font les outils

- Docker : build et run de l'image
 - Kubernetes : orchestration (YAML)
- Le client K8s Python existe (operators), hors de notre sujet

L9

LAB 9

Beacon en conteneur

45 min · auto-rythmé · socle + extension

Beacon en conteneur

SOCLE - pour tous

- Écrire un Dockerfile pour Beacon
 - Construire l'image et la versionner
 - Exécuter : config en volume, secret en env
- Vérifier le code de sortie du conteneur

EXTENSION - pour aller plus loin

- Build multi-stage, image finale slim
 - Utilisateur non-root
 - Ajouter un .dockerignore
- Comparer la taille des images obtenues



BLOC 3.2

Observabilité

Rendre Beacon observable

Un service en production ne doit pas seulement tourner : il doit pouvoir être interrogé sur ce qu'il fait. Trois signaux, complémentaires.



Logs

Événements datés et structurés : quoi, quand, dans quel contexte.



Métriques

Valeurs numériques agrégables : compteurs, latences, taux d'échec.



Traces

Cheminement d'une opération de bout en bout (survol - OpenTelemetry).

Logging structuré, pas de print

```
import logging

log = logging.getLogger("beacon")

log.info("probe terminée", extra={
    "target": t.name, "status": r.status,
    "ms": r.latency_ms})
```

Pourquoi structuré

- Niveaux et destinations gérés (vs print)
- Format JSON : parsable par la stack de logs

Jamais de secret dans un log

Exposer des métriques

```
from prometheus_client import (
    Counter, Histogram, start_http_server)

PROBES = Counter("beacon_probes_total",
                 "Sondes", ["status"])

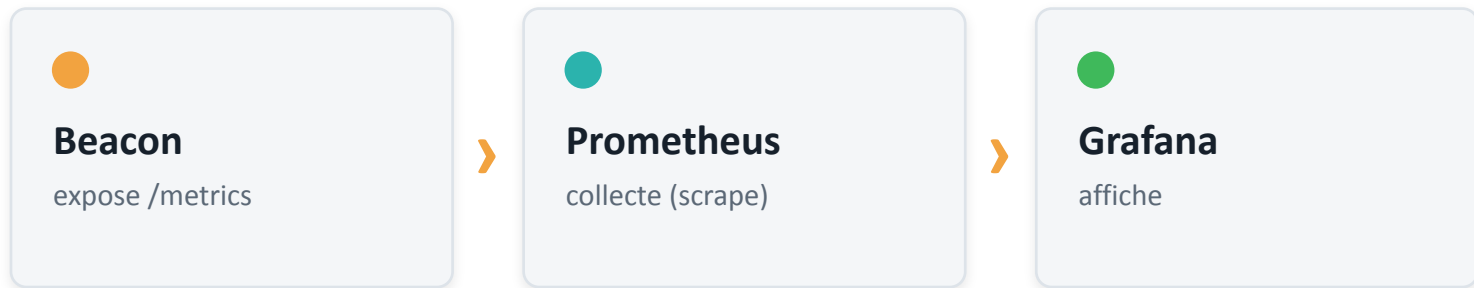
# dans la sonde
PROBES.labels(status=r.status).inc()
start_http_server(9100) # /metrics
```

Types de métriques

- Counter : ne fait que croître (sondes, échecs)
 - Histogram : distribution (latence)
- Gauge : valeur instantanée (cibles up)

On expose, on ne « dashboarde » pas

Beacon n'affiche pas de graphique. Il expose des métriques brutes ; des outils dédiés les collectent et les visualisent. Chacun son rôle.



Construire un dashboard Grafana en Python serait réinventer l'outil. Python s'arrête à l'exposition.

Beacon observable, en pratique

Métriques exposées

- beacon_probes_total{status}
- beacon_latency_seconds (histogram)
- beacon_targets_up (gauge)

Exposées sur /metrics

Logs structurés

- Un événement par sonde, au format JSON
- Cible, statut, latence en champs
- Niveaux info / warning / error

Aucun secret journalisé

Métriques en multi-process — ouverture

`start_http_server()` lance un thread HTTP en arrière-plan. Parfait pour une CLI ou une sonde long-running monolithique : exactement le cas de Beacon. La nuance n'apparaît que sous un serveur multi-process.

Le cas de Beacon (monolithique)

- Un seul process : un thread métriques
 - Compteurs cohérents par nature
- L'approche du cours est la bonne

Si un jour multi-process (Gunicorn...)

- Chaque worker a ses propres compteurs → métriques incohérentes
- Activer le mode multiprocess de `prometheus_client`
Compteurs partagés via `PROMETHEUS_MULTIPROC_DIR`

Pour une sonde périodique, le multiprocess serait une complexité inutile.

L10

LAB 10

Beacon observable

45 min · auto-rythmé · socle + extension

Beacon observable

SOCLE - pour tous

- Remplacer les print par du logging structuré
 - Définir un Counter et un Histogram
 - Les alimenter à chaque sonde
- Exposer un endpoint /metrics et le vérifier

EXTENSION - pour aller plus loin

- Labels pertinents (par cible, par type)
 - Une Gauge pour les cibles up
 - Formatter JSON pour les logs
- Survol OpenTelemetry (traces)

Avant la partie 2

Resynchronisation sur la solution de référence avant le bloc CI de la partie 2.

Beacon, à ce stade :

- Tourne en conteneur, livré comme une image versionnée
 - Reçoit config et secret au runtime, sans rien figer
 - Émet des logs structurés exploitables
- Expose un /metrics prêt à être collecté par Prometheus

M3 - CI & serverless

Acquis sur cette partie

- Image conteneur de Beacon
- Bonnes pratiques d'image
- Logs structurés

Métriques exposées

La prochaine partie

- Pipeline CI : lint, types, tests, build
- Automatiser la qualité et la livraison
- Serverless (optionnel)

Clôture et parcours « et après »



Fin de la partie 1

Beacon est conteneurisé et observable. La prochaine partie, un pipeline CI automatise sa qualité et sa livraison - la dernière brique.



PYTHON POUR LE DEVOPS

Module 3 · Partie 2

CI & serverless

Beacon entre dans une chaîne automatisée - et la formation se referme

Au programme - M3 partie 2

3.3

Intégrer dans une chaîne CI

Lint, types, tests, build d'image

50 min

L11

Lab - Beacon dans la CI

Socle + extension, auto-rythmé

45 min

3.4

Serverless (optionnel)

Fonction planifiée, bonnes pratiques

40 min

L12

Lab - Beacon serverless (option)

Soupe : selon le rythme du groupe

40 min



Clôture

Rétrospective, parcours « et après »

20 min

Beacon, après la partie 1

Acquis jusqu'ici

- Sonde réseau et cloud, secrets maîtrisés
- Packagé, testé, déployable via Ansible
- Conteneurisé en image versionnée

Observable : logs et /metrics

Objectif de la partie 2

- Automatiser qualité et livraison en CI
- Construire l'image à chaque changement
- Découvrir le serverless (optionnel)

Faire le bilan et tracer la suite

Fermer la boucle

La tension

La qualité tenue par la seule discipline de chacun ne passe pas à l'échelle d'une équipe.

Ce que cette demi-journée y répond

Automatiser les contrôles et la livraison en CI, refermer le fil rouge, puis prendre du recul sur le chemin parcouru.

À la fin de la partie 2

- 1 Comprendre la CI comme le « Ops » de DevOps.
- 2 Brancher lint, types et tests dans un pipeline.
- 3 Déclencher la chaîne à chaque push.
- 4 Construire l'image de Beacon automatiquement.
- 5 Situer le serverless et y porter une sonde (optionnel).
- 6 Faire le bilan de Beacon et identifier les suites.



BLOC 3.3

Intégrer dans une chaîne CI

La CI, c'est le « Ops »

Jusqu'ici, la qualité reposait sur la discipline de chacun. La CI l'automatise : à chaque push, la machine vérifie et construit. C'est ce qui transforme du bon code en livraison fiable.



Filet collectif

Personne ne casse la base sans que la CI le signale.



Vérité partagée

Les mêmes contrôles pour toute l'équipe, à chaque commit.



Livraison répétable

L'image se construit toujours de la même façon.

Lint, types, tests - déclarés une fois

```
[tool.ruff]
line-length = 100
```

```
[tool.mypy]
strict = true
```

```
# pytest découvre tests/ tout seul
```

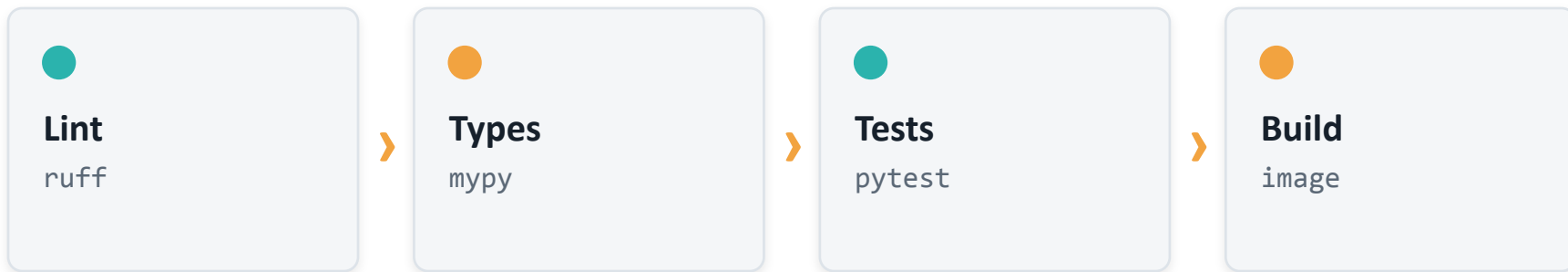
Trois contrôles

- ruff - lint et format, très rapide (Rust)
- mypy - typage statique, bugs attrapés tôt
- pytest - la suite du M1, en mocks

Tout vit dans pyproject.toml : une seule source de configuration, locale comme en CI.

Quatre étapes, à chaque push

Chaque étape conditionne la suivante : si le lint échoue, rien ne va plus loin. Un seul vert au bout = un changement livrable.



Échec à n'importe quelle étape = pipeline rouge. La qualité devient une condition, pas une intention.

Un workflow GitHub Actions

```
on: [push]
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -e ".[dev]"
      - run: ruff check .
      - run: mypy beacon
      - run: pytest
```

Portable

- GitLab CI : .gitlab-ci.yml, mêmes étapes (stages + script)
- Le principe ne dépend pas du fournisseur

Les commandes sont celles lancées en local

Construire l'image dans la CI

```
# après le vert lint + types + tests
- run: |
    docker build \
      -t beacon:$GITHUB_SHA .

# sur un tag de version : push au registre
    docker push beacon:$GITHUB_SHA
```

La boucle se ferme

- L'image de la partie 1, construite automatiquement
 - Tracée par commit (SHA)
- Prête à déployer, sans étape manuelle

Architectures cibles : build multi-plateforme

docker build produit une image pour l'architecture de la machine qui construit. Sur des postes variés, « ça build chez moi » ne garantit pas « ça tourne en prod ».

Le risque

- Poste Apple Silicon → image arm64
 - Runner CI ubuntu-latest → image amd64
- Cible d'une autre arch → ne démarre pas (ou émulation lente)

Règle pratique : en CI, fixer explicitement la plateforme de build pour qu'elle corresponde à la cible de déploiement, et passer en multi-arch si les cibles diffèrent.

```
# plateforme cible explicite
docker buildx build \
  --platform linux/amd64 \
  -t beacon:0.1 .
```

```
# image multi-architecture
docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t beacon:0.1 --push .
```


L11

LAB 11

Beacon dans la CI

45 min · auto-rythmé · socle + extension

Beacon dans la CI

SOCLE - pour tous

- Configurer ruff et mypy dans pyproject
- Écrire un workflow déclenché au push
- Enchaîner lint -> types -> tests

Vérifier le pipeline rouge puis vert

EXTENSION - pour aller plus loin

- Étape de build (et push) de l'image
- Matrice de versions de Python
- Cache des dépendances

Couverture publiée en artefact



BLOC 3.4 · OPTIONNEL

Serverless

Serverless : un serveur en moins

Une fonction serverless s'exécute sur déclenchement, sans serveur à provisionner ni maintenir, facturée à l'usage. Pour une sonde courte et périodique, c'est un bon candidat.



Pas de serveur

Ni provisioning ni maintenance d'OS - le fournisseur gère.



Déclenché

Sur planning (toutes les N min) ou sur événement.



Adapté à Beacon

Une sonde planifiée tient parfaitement dans une fonction.

Une sonde dans une fonction

```
def handler(event, context):  
    targets = load_config(CONFIG_PATH)  
    results = [probe_http(t)  
               for t in targets]  
    publish(results)  
    return {"checked": len(results)}
```

Le dividende du M1

- load_config et probe_http réutilisés tels quels
- Le code structuré paie ici : seul le point d'entrée change

Le cœur de Beacon reste le même

Réussir une fonction serverless

Cold start

Garder les dépendances légères ; initialiser hors du handler.

Packaging des deps

Embarquer les dépendances (zip ou layer) de façon reproductible.

Idempotence

Une réexécution ne doit pas produire d'effet de bord indésirable.

Secrets & logs

Secrets via la config de la fonction ; logs structurés vers le fournisseur.

L12

LAB 12 • OPTIONNEL

Beacon serverless

40 min · soupape selon le rythme du groupe

Beacon serverless

SOCLE - pour tous

- Écrire un handler réutilisant le cœur de Beacon
- Le déployer comme une fonction
- Le déclencher périodiquement (planning)

Vérifier les logs d'exécution

EXTENSION - pour aller plus loin

- Packaging propre des dépendances
- Secrets via la configuration de la fonction
- Publier le résultat (file, stockage, alerte)

Mesurer l'impact du cold start



CLÔTURE

Rétrospective & suites

De « ça marche chez moi » à un service livré

M1

Python opérationnel

Config, sonde locale, CLI, packaging, tests

M2

Réseau, cloud, infra

HTTP, secrets, SDK cloud, Ansible

M3

Production

Conteneur, observabilité, CI (+ serverless)

Un même fil rouge, Beacon, porté du script jetable au service packagé, testé, déployé, observé et intégré en CI.

Ce que la formation n'a pas couvert

Cette formation pose les fondations de Python pour le DevOps - pas l'exhaustivité. Quelques directions pour continuer, en connaissance de cause.

 Kubernetes & operators

L'orchestration en profondeur - et le client Python pour l'étendre.

 Concurrence avancée

asyncio à grande échelle pour des milliers de sondes parallèles.

 Terraform / IaC

Le provisioning déclaratif de l'infrastructure, complément du SDK.

 Tracing distribué

OpenTelemetry complet : relier logs, métriques et traces.

Ressources & prochaines étapes

Documentation

- httpx, prometheus_client, boto3
 - Ansible, Docker, GitHub Actions
 - ruff, mypy, pytest
- Packaging : pyproject, uv

Continuer avec Beacon

- Reprendre le code comme base de projet
 - Ajouter une sonde de son choix
 - Le brancher à un vrai Prometheus
- Le déployer sur son infra



Merci

Python n'est pas l'outil unique du DevOps - c'est un excellent langage de logique et de colle, à sa juste place. Beacon en est la preuve, de bout en bout.